

AEM as a Cloud Service Best Practices

All Roles – SHRSS-Aware Consolidated Volume

Purpose A deep, experience-based reference for AEM as a Cloud Service (AEMaaCS) across all roles (engineering, admin/DevOps, authors, DAM). This volume goes beyond Experience League in depth and opinion, while continuously pointing to public docs for alignment. SHRSS-specific overlays highlight how these practices apply to the Seminole Hard Rock Support Services implementation.

1. How to Use This Volume

- **For engineering (developers & technical architects)** Focus on §§2–3 and the *Engineering SHRSS Overlay* subsections.
- **For admins / DevOps** Focus on §§2 and 4 plus the *Admin/DevOps SHRSS Overlay*.
- **For authors** Focus on §5 plus the *Author SHRSS Overlay*; pair with the SHRSS AEM Content Authoring Guide.
- **For DAM architects/librarians** Focus on §6 plus the *DAM SHRSS Overlay*.
- **For indexing & performance** Use this volume as an overview; see the separate **SHRSS Indexing & Performance Volume** for worked examples (e.g., Jobs index).

Cross-link this volume from your role-specific volumes and from the SHRSS SDD as “Authoritative AEMaaCS Practices.”

2. Platform & Architecture Fundamentals

2.1 Cloud-Native Principles

Key ideas

- **Immutable + stateless code**
 - Application code is baked into images; authors never edit /apps or /libs in runtime.
 - Runtime instances are stateless; any state must live in JCR, external systems, or caches with safe invalidation.
- **Horizontal scale, not vertical**
 - Scale is achieved by adding pods, not by making servers bigger.
 - Code must be **idempotent and cluster-safe**—no “singleton” assumptions.
- **Always-on, always-current**
 - No more one-off “upgrade projects”: AEMaaCS is continuously updated by Adobe.
 - Custom code must tolerate underlying platform changes and follow supported APIs.

Reference

- [Introduction to the Architecture of Adobe Experience Manager as a Cloud Service](#)
- [AEM Technical Foundations](#)

SHRSS Overlay – Platform

- SHRSS uses **AEM Sites + Assets** on AEMaaCS for jobs, events, news, locations, careers, and brand experiences.
- Most **business data** (Jobs, Events, Locations, some Shared Data) is modeled as **Content Fragments** (CFs) under `/content/dam/shrss/...`
- Integrations (e.g., **Workday** → **Jobs**, DPLT → Locations, TransPerfect → translations) treat AEM as a **view + orchestration layer**, not a system of record.
- SHRSS pipelines and RDEs must treat **every environment as ephemeral**: no manual config, no environment-only code, no author-side hotfixes in `/apps`.

3. Engineering (Developers & Technical Architects)

3.1 Core Engineering Practices

3.1.1 Project & Module Structure

- Use the **AEM Project Archetype** patterns:
 - `core`: Java, Sling Models, OSGi services, schedulers.
 - `ui.apps`: components, dialogs, clientlibs, policies.
 - `ui.frontend`: front-end build (Webpack/Rollup), compiled into clientlibs.
 - `ui.config`: OSGi configs, repo init.
 - `dispatcher`: Apache/Dispatcher configuration for cloud.
 - `it.tests` / `ui.tests`: integration and Cypress/Playwright tests wired to Cloud Manager.
- **Never ship content** (e.g., real pages, users, groups) in `ui.apps`; limit `ui.content` to baseline configuration/content only.

References

- [AEM Project Structure](#)
- [AEM Project Archetype Overview](#)

3.1.2 Components, HTL & Sling Models

- **HTL (Sightly) best practices**
 - Zero business logic: only simple expressions, conditionals, iteration.
 - Delegate to Sling Models for any logic or external integrations.

- **Sling Models**
 - Prefer `@Model(adaptables = Resource.class)` for components; `SlingHttpServletRequest` only when needed.
 - Keep models pure: no writes to repository in getters; no heavy I/O in constructors.
- **OSGi Services**
 - Encapsulate integrations (Workday, TransPerfect, DPLT, etc.) behind interfaces.
 - Use **factory configs** and environment variables for endpoints/credentials, never hard-code URLs or secrets.

References

- [Component Development in AEM Sites](#)

3.1.3 Cloud-Aware Coding

- **Cluster awareness**
 - Schedulers and listeners must be idempotent and safe when executed on multiple pods.
 - Avoid in-memory caches as primary state; if you must cache, use short TTL and treat them as hints.
- **I/O discipline**
 - No writes to local filesystem except officially supported temp locations (rare).
 - All durable state goes into JCR or external systems (e.g., Workday).
- **Indexing awareness**
 - Design queries before writing components.
 - Ensure every non-trivial query is backed by an appropriate index rule (or OOTB index).

References

- [AEM as a Cloud Service Development Guidelines](#)
- [Query and Indexing Best Practices](#)

SHRSS Overlay – Engineering

- **Content Fragments as primary data model**
 - Jobs, Events, News, and Locations are modeled as CFs with **integration-friendly fields** (IDs, URLs, dates, references).
 - SHRSS Sling Models should treat CFs as **immutable records** coming from Workday/DPLT; only enrich with rendering-specific metadata.
- **Workday Jobs integration**
 - Workday drives core job fields; SHRSS code must:

- Treat Workday IDs as **immutable foreign keys**.
- Respect the “Is API Data” flag: API-managed vs author-managed jobs follow different update rules.
- When building queries or indexes (e.g., Jobs listing/search), **always include the CF model path** and/or contentFragment=true in constraints.
- **GraphQL & persisted queries**
 - For headless or component-level data access (e.g., Jobs/Events lists), SHRSS favors **persisted GraphQL queries** with caching through CDN/Dispatcher.
 - Engineering should:
 - Keep persisted queries **versioned** and named by use case (e.g., jobsByLocation_v1).
 - Avoid sending arbitrary POST queries from front-end in production; use GET persisted queries instead.
- **Dispatcher/CDN alignment**
 - SHRSS list/detail components are designed to be **cache-friendly**:
 - No user-specific state in Jobs/Events/News listing responses.
 - Clear cache keys and TTLs to avoid stale results after CF updates.
 - Engineering must honor these constraints when evolving components (e.g., avoid mixing personalization with heavily cached templates).

4. Admin / DevOps

4.1 Cloud Manager & Environments

- **Programs & environments**
 - Use a simple, predictable env set: DEV → STAGE → PROD, plus **RDE** for rapid dev testing.
 - Keep environment purpose clear: DEV for integration, STAGE for release validation, PROD for live traffic.
- **Pipelines**
 - Non-prod pipeline: for DEV deployments and quality checks.
 - Prod pipeline: Stage + Prod deployment with quality gates, functional tests, UI tests, and performance/experience checks.
- **Quality gates**
 - Treat code quality, security, and performance scores as **blocking** unless there is a documented, reviewed exception.

References

- [CI/CD Pipelines](#)
- [Using the CI/CD Pipeline in Cloud Manager](#)

4.2 Operations, Monitoring & Troubleshooting

- **Logging**
 - Prefer Cloud Manager logs for historical analysis; Developer Console for live debugging.
- **Runtime health**
 - Use Developer Console health checks and status endpoints; never SSH into hosts.
- **Rollback & content integrity**
 - Roll back via **code redeploy** (previous image), not via content manipulation.
 - For content issues, use **content restore** and **content copy** tools, not manual JCR edits.

References

- [Debugging AEM as a Cloud Service with the Developer Console](#)
- [Troubleshooting AEM as a Cloud Service](#)

SHRSS Overlay – Admin/DevOps

- **Program & pipeline strategy**
 - SHRSS uses a standard three-tier environment topology plus RDEs; ensure:
 - **Jobs & Events indices** are stable before promoting code.
 - Integration endpoints (Workday, TransPerfect, DPLT) are configured via environment variables/OSGi configs, not hard-coded.
- **Search/indexing monitoring**
 - SHRSS relies heavily on Jobs/Events/News search; admins should:
 - Watch for “**query without index detected**” alerts.
 - Use the **Query Performance Tool/Query Analyzer** to periodically review top queries (e.g., Jobs search by location).
- **Dispatcher rules**
 - Filtering rules must protect SHRSS custom servlets and APIs under /bin, /api, etc.
 - Any change in SHRSS integration endpoints or GraphQL endpoints must be reviewed for security and caching impact.

5. Authors

5.1 General Authoring Practices

- Prefer **Core Components** and SHRSS components built on them; avoid custom HTML if a component exists.
- Keep **content and presentation separate**:
 - Use CFs and tags for data, templates and policies for look & feel.

- Rely on **language copies and MSM** where appropriate; avoid manual duplication across locales.

References

- [Authoring in AEM Sites](#)
- [Content Fragments](#)

SHRSS Overlay – Authors

- **Jobs**
 - Jobs are CFs under Assets > Files > SHRSS > Content Fragments > Jobs > <region>/<country>/<property>.
 - Follow the **“Is API Data”** flag rule:
 - ON → Workday job; do not change IDs or system fields.
 - OFF → manual job; maintain tag consistency with Workday jobs.
 - Use **Hot Job** flag sparingly to keep visual emphasis meaningful.
- **Events**
 - Model events as CFs referencing **Location CFs** for consistency in addresses and maps.
 - Use event status + status message fields to manage cancellations/rescheduling instead of editing titles.
- **News**
 - Use CFs for structured news metadata (date, category, hero image) and pages/components for narrative layout.
- **Locations**
 - Locations come from DPLT; treat **Location CFs as authoritative** for address/geo data.
 - If you need to correct a location, follow the agreed DPLT change process; do not “fix” addresses directly in AEM.
- **Navigation & Data Display**
 - Use shared data CFs and tags to drive **menus, cards, and filters** consistently across Jobs, Events, and News.
 - When something does not appear in a list:
 - Check CF is **published**.
 - Check tags and required fields are set.
 - Check effective dates (for Jobs/Events).

Pair this with the SHRSS AEM Content Authoring Guide for step-by-step “how to” instructions.

6. DAM Architects / Librarians

6.1 Asset & CF Governance

- Design folder structures around **business domains & access**, not system tables.
- Normalize metadata: required fields, controlled vocabularies (tags), approval flows.
- Use **Content Fragments for structured data** (Jobs, Events, Locations) and **Assets for media** (images, docs, video).

References

- [Architecture of Assets as a Cloud Service solution](#)
- [Assets as a Cloud Service documentation](#)

SHRSS Overlay – DAM

- **Folder IA**
 - CFs live under Assets > Files > SHRSS > Content Fragments > ...
 - Media assets under Assets > Files > SHRSS > Media > ...
 - Keep CF folders aligned with **Jobs/Events/Locations** hierarchies (e.g., region/country/property).
- **Metadata & tags**
 - Maintain tag taxonomies that align with **Workday (jobs)** and **DPLT (locations)**; never create ad-hoc tags that break filters.
 - Use metadata schemas to:
 - Enforce required fields for job images, event hero images, etc.
 - Ensure rights/expiry metadata is present for marketing assets.
- **Lifecycle**
 - Define lifecycle for:
 - Expired jobs → archived CFs; related media kept or archived based on reuse.
 - Past events → archived CFs; long-tail SEO pages may remain live but marked as past.

7. Cross-Cutting: Indexing & Performance

- Treat **index design as part of the feature design**, not as an afterthought:
 - Always design queries before building components.
 - Ensure each non-trivial query has a backing index or leverages an OOTB one.
- Use **property + ordered properties** for filters & sorting, rather than fulltext over everything.
- Guard against:

- LIKE '%foo%' patterns without proper index rules.
- Queries without ISDESCENDANTNODE constraints.
- Over-indexing (indexing every property) which inflates index size and slows reindex.

References

- [Content Search and Indexing](#)
- [Search and indexing in AEM as a Cloud Service](#)

SHRSS Overlay – Indexing & Performance

- Jobs, Events, News, and Locations rely on **CF-backed searches**; for each domain:
 - Queries must constrain by:
 - Path (e.g., /content/dam/shrss/content-fragments/jobs),
 - CF model path (/conf/shrss/.../models/job),
 - contentFragment=true.
 - Indexes should be **customizations of OOTB Lucene indexes** (e.g., damAssetLucene-*) following *-custom-* naming patterns.
- The **SHRSS Indexing & Performance Volume** contains:
 - A worked example of a Jobs index definition (damAssetLucene-8-custom-*) and line-by-line explanation.
 - Guidance for using **Query Performance Tool, Query Analyzer, and Developer Console** to identify slow queries impacting Jobs/Events listings.

8. Governance & Release Management

- Keep **branching strategy** simple and visible:
 - main → production pipeline.
 - develop → non-prod pipeline.
 - Feature branches → short-lived, PR-driven.
- Require:
 - Automated testing (unit + IT + UI) for changes touching shared components.
 - Index review for features introducing new queries or filters.
- Run periodic **technical debt reviews**:
 - “Code smells” (tight coupling, hard-coded paths).
 - Config drift across environments.
 - Overgrown indexes or dead feature flags.

9. References

Core public references used throughout this volume:

- [AEM Technical Foundations](#)
- [Introduction to the Architecture of Adobe Experience Manager as a Cloud Service](#)
- [Implementing Applications for AEM as a Cloud Service](#)
- [AEM as a Cloud Service Development Guidelines](#)
- [Content Search and Indexing](#)
- [Query and Indexing Best Practices](#)
- [Cloud Manager – Key Concepts](#)

Use these links as canonical anchors when evolving this volume.